Computer Vision and Deep Learning

Lab 02: Image classification

Today

- Introduction to PyTorch
- Recap of the image classification task
- Multi-class image classification on the Fashion-MNIST dataset using a ResNet-18
- Binary image classification on the DogsVsCats dataset using a ResNet-18

PyTorch

Python package that provides tensor computation

- Tensor computation (like Numpy) with strong GPU acceleration
- Deep neural networks built on an autograd system

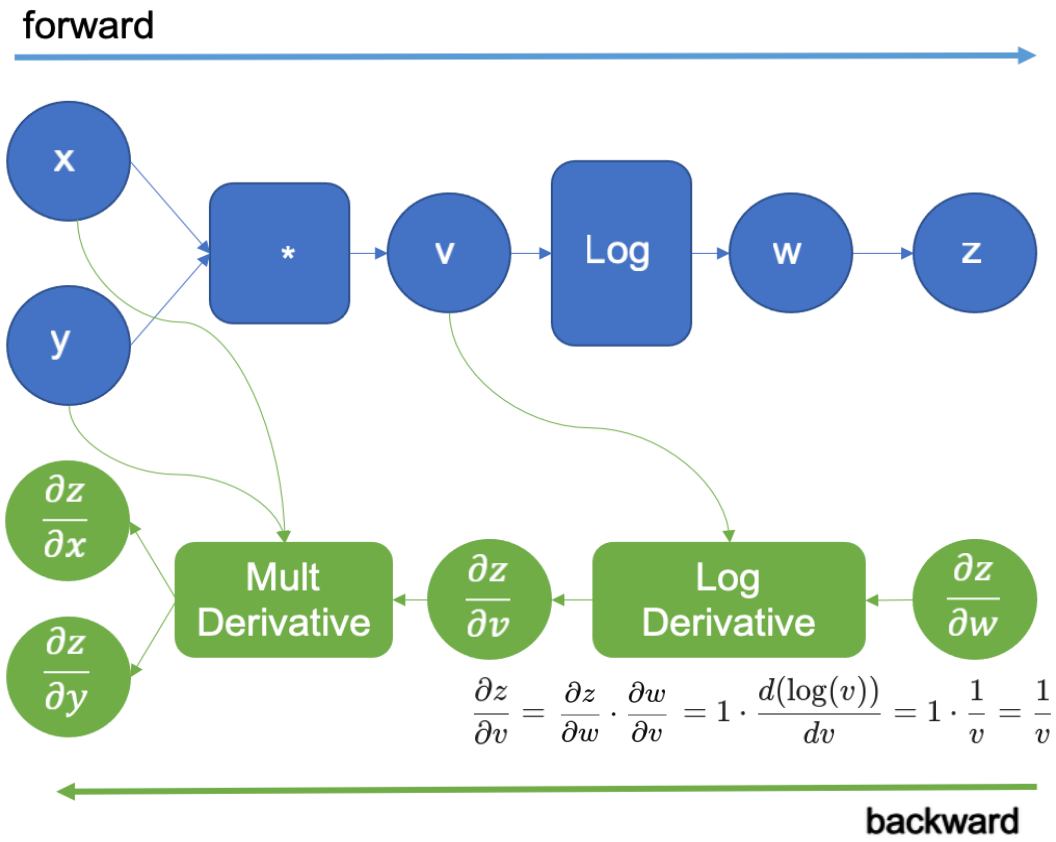
(*) https://github.com/pytorch/pytorch

PyTorch: Autograd Engine

```
$ import torch
$ x = torch.randn((1,), requires_grad=True)
$ y = torch.randn((1,), requires_grad=True)
$ v = x*y
$ z = torch.log(v)
$ z.backward()
$ x.grad
```

$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial x} = \frac{1}{v} \cdot \frac{\partial (x \cdot y)}{\partial x} = \frac{1}{v} \cdot y$$

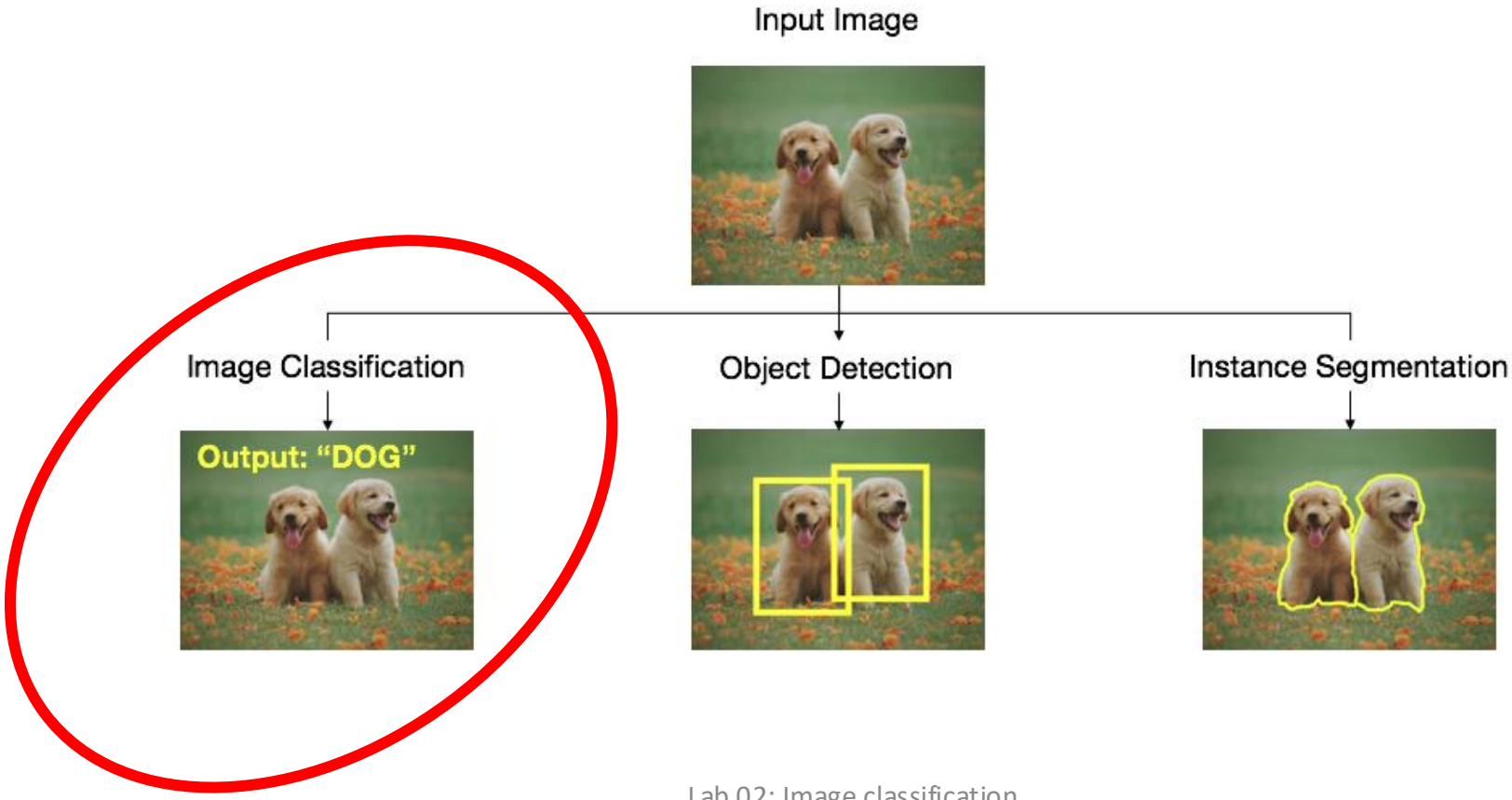
$$\frac{\partial z}{\partial y} = \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial y} = \frac{1}{v} \cdot \frac{\partial (x \cdot y)}{\partial y} = \frac{1}{v} \cdot x$$



(*) https://pytorch.org/blog/overview-of-pytorch-autograd-engine/

Image classification

- ***Image classification*** is a core task in CV



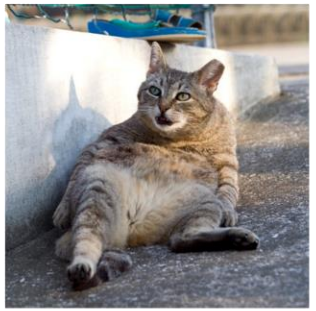
The challenges

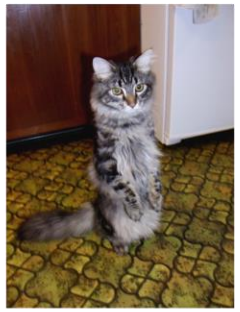
Background clutter

Illumination

Occlusion

Deformation



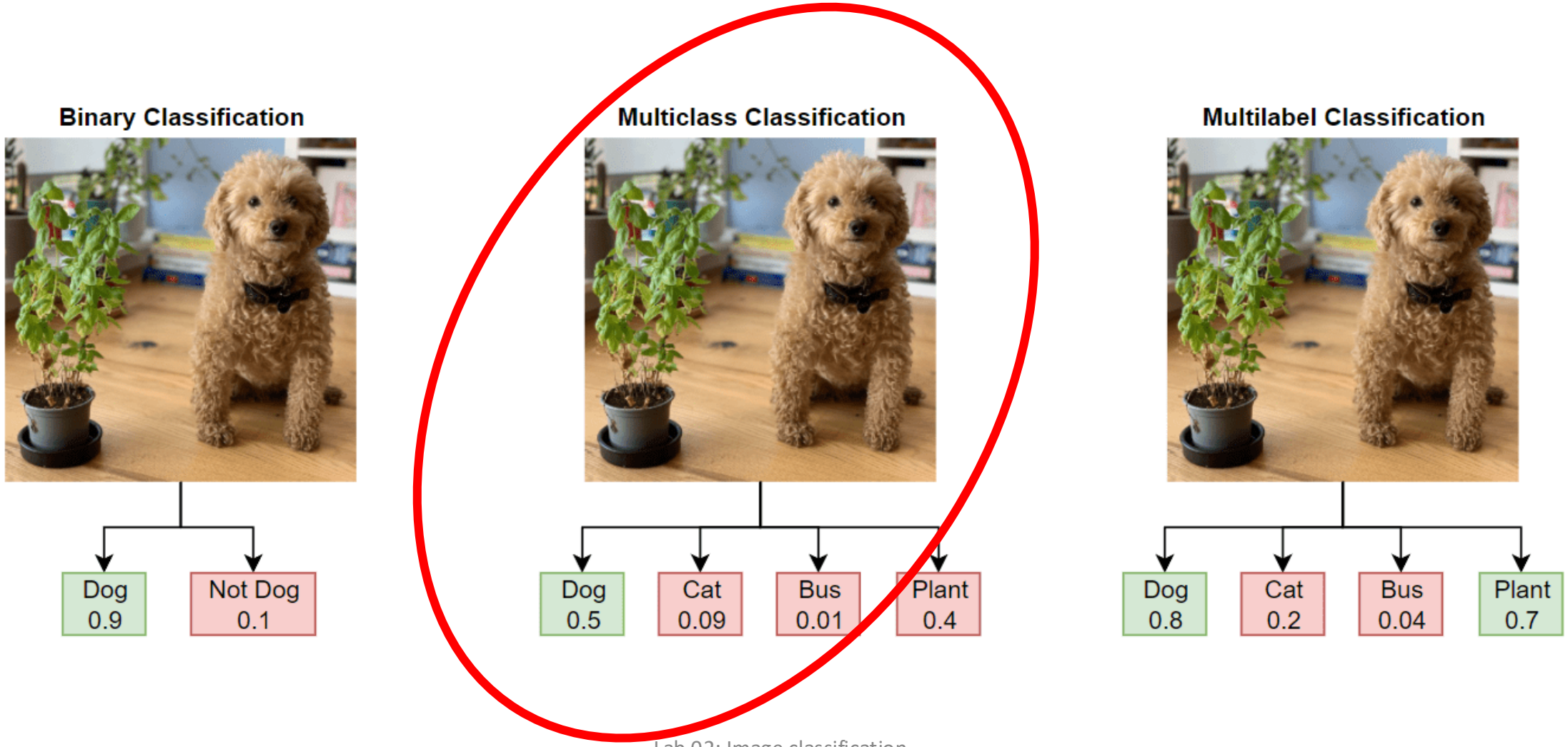


The DL-based data driven approach

1. Collect a dataset of images and labels
2. Use a DNN to train a classifier
3. Evaluate the classifier on new images

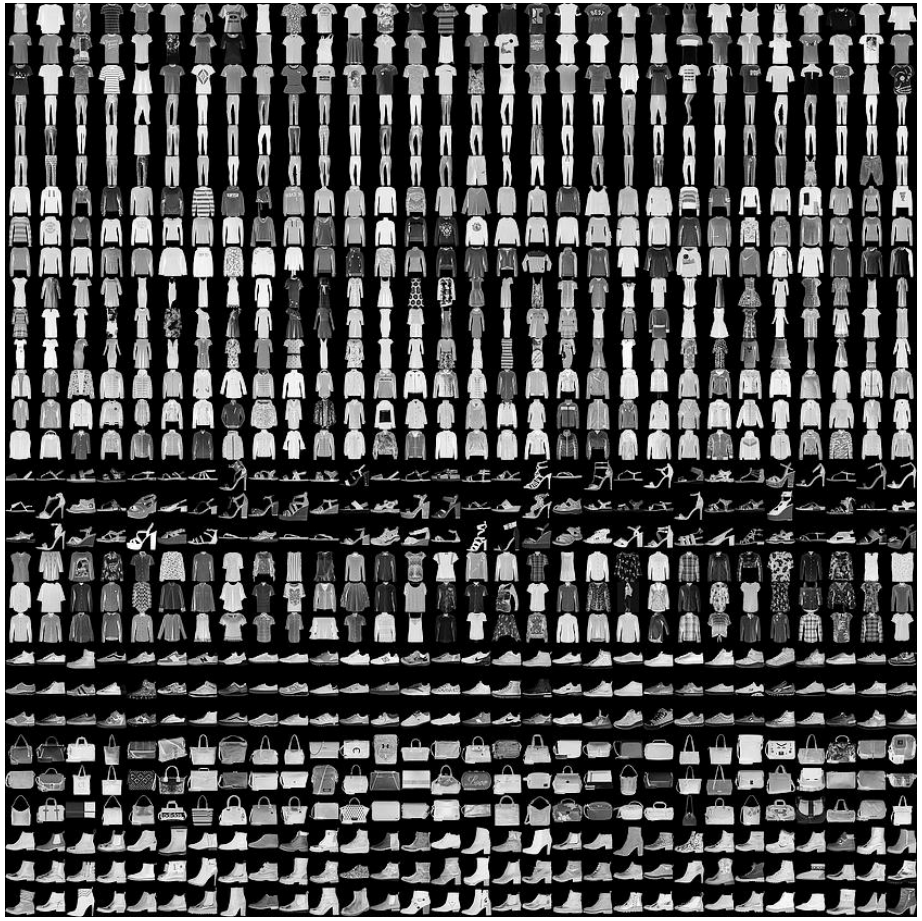
Multi-class image classification on the Fashion-MNIST dataset using a ResNet-18

Binary vs multiclass vs multilabel



Fashion-MNIST

- Fashion-MNIST is a dataset of Zalando's article images
- It consists of a training set of 60,000 examples and a test set of 10,000 examples
- Each example is a 28x28 grayscale image associated with a label from 10 classes



(*) https://github.com/zalandoresearch/fashion-mnist

ResNet-18

- Generally, deeper neural networks face the issue of vanishing gradients
- The ***ResNet*** neural networks ***introduced the concept of skip connections.*** This, allows us to create deeper networks ***without worrying about vanishing gradients***
- In the original paper[1], 5 ResNet models were presented. They were ResNet-18, ResNet-34, ResNet-50, ResNet-101, and ResNet-152

(1) https://arxiv.org/pdf/1512.03385.pdf

ResNet-18 (cont'd)

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 64 \\ 3\times3, 64 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 64 \\ 3\times3, 64 \\ 1\times1, 256 \end{bmatrix}\times3$
conv3_x	28×28	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 128 \\ 3\times3, 128 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times4$	$\begin{bmatrix} 1\times1, 128 \\ 3\times3, 128 \\ 1\times1, 512 \end{bmatrix}\times8$
conv4_x	14×14	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 256 \\ 3\times3, 256 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times6$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times23$	$\begin{bmatrix} 1\times1, 256 \\ 3\times3, 256 \\ 1\times1, 1024 \end{bmatrix}\times36$
conv5_x	7×7	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times2$	$\begin{bmatrix} 3\times3, 512 \\ 3\times3, 512 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$	$\begin{bmatrix} 1\times1, 512 \\ 3\times3, 512 \\ 1\times1, 2048 \end{bmatrix}\times3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Step 1: DNN setup w.r.t our task

- Create a custom `FMnistResNet18` class in which:
  - Load a pre-trained ResNet-18 model
  - Change the first and last layers
- Specifically, the input and output layers of a pre-trained ResNet-18 need to be changed, since ResNet was originally designed for ImageNet competition, which was a color (3-channel) image classification task with 1000 classes
- Fashon-MNIST, on the other hand, only contains 10 classes, and its images are in the grayscale (*i.e.*, 1-channel)

Step 2: DataLoaders and model

- Create DataLoaders
  - Hint: https://pytorch.org/tutorials/beginner/basics/data_tutorial.html
- What does a PyTorch dataloader do?
  - ***Define a dataset to work with***: identifying where the data is coming from and how it should be accessed
  - ***Batch the data***: define how many training or testing samples to use in a single iteration
  - ***Shuffle the data***: PyTorch can handle shuffling data for you as it loads data into batches
  - ***Load data directly on CUDA tensors***: because PyTorch can run on the GPU, you can load the data directly onto CUDA before they're returned
- Define the model

Step 3: Loss and optimizer

- Define the loss
- What is Cross Entropy loss?
  - Measures the performance of a classification model whose output is a probability value between 0 and 1
  - It increases as the predicted probability diverges from the actual label
  - A perfect model would have a log loss of 0
- Define the optimizer

$$l_o = -\sum_{c=1}^{M} y_{o,c} \log(p_{o,c})$$

- M: the number of classes
- log: the natural logarithm
- y: binary indicator (0 or 1) if the class label c is the correct classification for observation o
- p: predicted probability that observation o is of class c

(*) https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html

Step 4: Training and Evaluation

- Write the training step
- Write the evaluation step
  - The base metric used for model evaluation is often ***accuracy***, describing the number of correct predictions overall predictions:

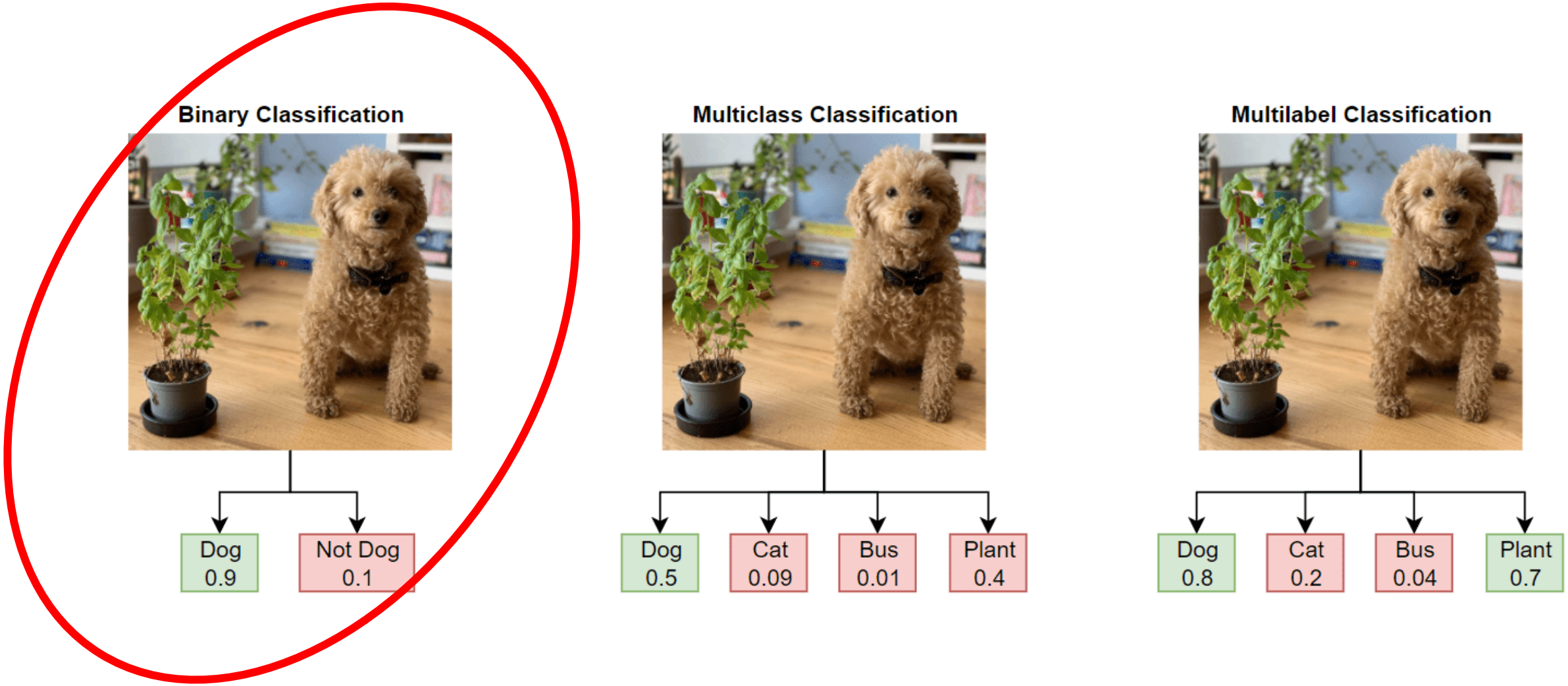
$$ACC = \frac{n.of\ correct\ prediction}{n.of\ all\ prediction}$$

- Accuracy may not be a good measure if the dataset is not balanced, using it can result in a misleading interpretation of results

(*) https://pytorch.org/tutorials/beginner/introyt/trainingyt.html

Binary image classification on the DogsVsCats dataset using a ResNet-18

Binary vs multiclass vs multilabel



Dogs vs cats dataset

- The Dogs vs Cats dataset is a standard computer vision dataset that involves classifying photos as either containing a dog or cat
- The Kaggle competition provided 25,000 labeled photos: 12,500 dogs and the same number of cats

(*) https://www.kaggle.com/competitions/dogs-vs-cats

Binary cross entropy loss

- ***Binary cross-entropy*** is a loss function that is used in binary classification problems
- When there are ***only 2 discrete quantities*** then this function ***is the best choice*** in DL

$$L = -\frac{1}{N}\sum_{i=1}^{N} y_i \log\big(p(y_i)\big) + (1 - y_i)\log\big(1 - p(y_i)\big)$$

(*) https://pytorch.org/docs/stable/generated/torch.nn.BCELoss.html

Precision, recall, and F1-Score

- Precision:

$$Precision = \frac{TP}{TP + FP}$$

- Recall:

$$Recall = \frac{TP}{TP + FN}$$

- F1-Score:

$$F1Score = 2 * \frac{Precision * Recall}{Precison + Recall}$$

(*) https://medium.com/analytics-vidhya/confusion-matrix-accuracy-precision-recall-f1-score

Exercises